

Claviger - Convention de docstrings et stratégie de documentation du code

Décision de projet - documentation sélective, contractuelle et maintenable

Statut : convention de projet - V1 candidate / préparation V1.1

Périmètre : `src/claviger/`

Objectif : préserver les contrats importants du code sans créer une documentation exhaustive, redondante et coûteuse à maintenir.

1. Décision de documentation

Claviger ne cherche pas à documenter manuellement chaque fonction du projet.

Le code, le typage, les noms des symboles, les tests et les outils d'analyse modernes expliquent déjà une grande partie du **comment**. Les docstrings doivent donc conserver ce que le code montre moins bien : le **contrat**, l'**intention**, les **invariants**, les **effets de bord**, les **erreurs significatives** et les **frontières architecturales**.

Une docstring utile doit répondre à au moins une question qu'un développeur ne peut pas résoudre immédiatement en lisant uniquement le nom et la signature.

Principe : documenter ce qu'il faut savoir avant de modifier ou d'appeler le composant, pas paraphraser son implémentation ligne par ligne.

Cette convention complète le README, les documents d'architecture et le guide pédagogique. Elle ne les remplace pas.

2. Pourquoi ne pas viser 100 % des fonctions

Une couverture documentaire de 100 % n'est pas un objectif de qualité en soi.

Une fonction telle que :

```
def has_changes(self) -> bool:
    return bool(self.add_role_ids or self.remove_role_ids)
```

n'a généralement pas besoin d'une longue docstring si son nom, son type de retour et son contexte suffisent.

À l'inverse, une méthode d'exécution peut nécessiter une documentation plus riche même si son code est court, parce qu'elle porte un contrat important :

- prévalidation avant la première mutation Discord ;
- ordre d'application des opérations ;
- effets de bord externes ;
- exceptions métier ;
- comportement en cas d'échec partiel.

La valeur documentaire est donc liée à la **complexité du contrat**, pas au nombre de lignes.

3. État constaté avant V1.1

Un comptage statique de la branche audité montre que le projet possède déjà une base documentaire importante :

- 139 fonctions ou méthodes détectées, dont 124 avec docstring ;
- 91 classes détectées, dont 90 avec docstring.

Ces chiffres ne constituent pas un score de qualité : ils incluent des fonctions internes et ne mesurent pas la précision des textes. Ils montrent cependant que le projet n'a pas un problème de docstrings absentes à grande échelle.

La priorité avant V1.1 n'est donc **pas** de remplir mécaniquement les quelques trous restants. Elle est de stabiliser la gestion des erreurs, le reporting et l'observabilité, puis de documenter les contrats définitifs de ces composants.

4. Langue et style

4.1 Langue

Les docstrings du code restent en **anglais**, conformément à la convention déjà dominante dans `src/claviger/`, aux identifiants du projet et à l'écosystème Python.

La documentation de haut niveau peut rester en français. Il n'est pas nécessaire que les deux couches utilisent la même langue.

4.2 Style général

Claviger suit une variante pragmatique de PEP 257 :

- première phrase courte décrivant la responsabilité ou le contrat ;
- phrase à l'impératif ou formulation directe ;
- une seule ligne lorsque cela suffit ;
- bloc plus détaillé uniquement si le comportement mérite d'être expliqué ;
- pas de répétition mécanique des types déjà fournis par les annotations ;
- pas de section `Args` / `Returns` systématique si elle n'apporte aucune information supplémentaire ;
- documenter `Raises` lorsque l'exception fait partie du contrat qu'un appelant doit connaître ;
- préciser les effets de bord et invariants importants.

Exemple court adapté au style actuel :

```
class Reporter(Protocol):
    """Define a destination capable of receiving structured reports."""
```

Cette docstring suffit parce que le `Protocol`, la signature de `report()` et le type `ReportEvent` expriment déjà le reste.

5. Quand une docstring est obligatoire

5.1 Classes constituant une frontière architecturale

Docstring requise pour les classes qui structurent l'application :

- services métier publics ;
- planners ;
- executors ;
- coordinators ;
- repositories ;
- politiques et resolvers ;
- reporters ;
- interfaces / `Protocol` ;
- composants de composition ou de cycle de vie lorsqu'ils jouent un rôle non évident.

La docstring doit expliquer **la responsabilité et la frontière**, pas lister toutes les méthodes.

Exemple :

```
class MemberRolePlannerService:
    """Plan member and interest role changes without touching Discord."""
```

La précision `without touching Discord` est importante : elle protège une propriété architecturale du composant.

5.2 Méthodes publiques avec contrat métier non trivial

Docstring requise lorsque l'appelant doit connaître une règle qui n'apparaît pas dans la signature.

Exemples :

- une méthode refuse des clés de questionnaire inconnues ;
- un executor valide tous les rôles avant la première mutation ;
- une synchronisation doit être exécutée dans une transaction partagée ;
- une policy bascule vers une configuration de repli restrictive ;
- une méthode distingue les métadonnées humaines des données observées sur Discord.

5.3 Effets de bord externes

Docstring fortement recommandée pour une opération qui :

- modifie Discord ;
- écrit dans SQLite ;
- publie un report ;
- modifie un état persistant ;
- appelle une API externe future ;
- déclenche une opération potentiellement destructive.

La docstring doit préciser **ce qui peut être modifié**, et non détailler chaque appel interne.

5.4 Exceptions qui font partie du contrat

Après le refactoring V1.1, les services concernés doivent documenter les exceptions métier qu'un appelant est censé traiter.

Exemple cible :

```
async def execute(...) -> MemberRoleExecutionResult:
    """Apply a prevalidated role plan to a Discord member.

    Raises:
        MemberRoleNotFoundError: A planned role no longer exists.
        MemberRoleNotManageableError: Discord no longer allows the bot to manage a role.
        PartialRoleExecutionError: Discord failed after at least one mutation was applied.
    """
```

Cet exemple décrit une **cible de contrat**. Il ne doit être ajouté au code que lorsque l'exception d'exécution partielle existe réellement.

5.5 Overrides et hooks dont le rôle n'est pas évident

Les hooks de framework peuvent mériter une docstring même lorsque leur nom vient de la bibliothèque.

Pour `ClavigerBot.setup_hook()`, une documentation courte est pertinente car ce hook est un point de cycle de vie important de `discord.py` : il synchronise les commandes de la guild cible avant le fonctionnement normal du bot.

Exemple :

```
async def setup_hook(self) -> None:
    """Synchronize guild application commands during bot startup."""
```

Pour `on_ready()`, une docstring est optionnelle si la méthode ne fait que produire une information de démarrage. Si elle reçoit plus tard de vraies responsabilités opérationnelles, son contrat devra être documenté.

6. Quand une docstring est optionnelle

Une docstring peut être utile, sans être obligatoire, pour :

- un helper privé avec un algorithme ou une convention subtile ;
- une propriété calculée portant une sémantique métier importante ;
- un adaptateur fin autour d'une API externe ;
- une fonction dont le nom est clair mais dont une contrainte de format n'est pas évidente ;
- une factory qui réalise plus qu'un simple assemblage.

Exemple : un helper qui limite un texte à la longueur autorisée par Discord mérite une docstring si la limite ou la conservation de l'ellipse fait partie du comportement attendu.

7. Quand ne pas écrire de docstring

Ne pas ajouter de docstring uniquement pour atteindre un pourcentage arbitraire.

Elle est généralement inutile pour :

- un getter trivial ;
- une propriété évidente dont le nom décrit exactement le calcul ;
- une petite fonction privée sans règle particulière ;
- une fonction de test dont le nom décrit déjà le scénario ;
- du boilerplate de construction ;
- une callback Discord dont le décorateur, le nom et la signature suffisent, sauf si elle porte un contrat particulier ;
- une méthode qui ne ferait que paraphraser le code.

Mauvais exemple :

```
def get_role(self, role_id: int) -> Role | None:
    """Get a role using the role ID."""
```

Si aucune autre règle n'existe, cette phrase n'apporte rien.

8. Contenu attendu d'une bonne docstring

Une bonne docstring peut contenir, selon le besoin :

Responsabilité

Ce que le composant est autorisé à faire.

```
"""Build the desired role mutations without performing Discord I/O."""
```

Invariant

Ce qui doit rester vrai même après un refactoring.

```
"""Apply one catalog sync plan without committing the transaction."""
```

Ici, `without committing` est une information architecturale capitale : le commit appartient au coordinator afin que plusieurs repositories participent à une même transaction.

Effet de bord

Ce que l'appel peut réellement modifier.

```
"""Publish one structured event as a Discord forum post."""
```

Condition d'échec

Une erreur métier qu'un appelant doit distinguer d'un résultat valide.

```
class UnknownMemberInterestError(ValueError):
    """Raised when a submitted interest is not part of the questionnaire."""
```

Raison d'une contrainte inhabituelle

Lorsqu'un comportement peut sembler surprenant, expliquer pourquoi il existe.

Exemple cible :

```
def build_plan(...):
    """Build the desired role changes from the latest questionnaire state.

    The planner only considers roles managed by this workflow so unrelated
    Discord roles remain untouched.
    """
```

9. Ce qu'une docstring ne doit pas contenir

Éviter :

- l'historique complet du refactoring ;
- des décisions qui appartiennent à un ADR ou à la documentation d'architecture ;
- des secrets, IDs réels ou noms de personnes ;
- des exemples de données sensibles ;
- une traduction ligne par ligne de l'implémentation ;
- des détails susceptibles de changer à chaque petite modification interne ;
- des promesses qui ne sont pas encore implémentées.

Exemple à éviter :

```
"""Loop over every role, call get_role, then remove roles before adding them."""
```

Cette phrase décrit l'implémentation actuelle. Si l'ordre interne change sans modifier le contrat, la docstring devient fautive inutilement.

10. Convention par type de composant Clavier

Type	Documentation attendue	Point principal à préserver
Model / dataclass	Courte, si la sémantique n'est pas évidente	Ce que représente l'objet
Planner	Oui	Décision pure, sans I/O Discord
Executor	Oui	Effets de bord, prévalidation, erreurs
Coordinator	Oui	Orchestration et limites de responsabilité
Repository	Oui pour la classe et opérations non triviales	Persistance, transaction, identité
Policy / Resolver	Oui	Règles de repli et autorisation
Reporter / Protocol	Oui	Contrat de destination et isolation des reporters
Command factory	Courte si utile	Assemblage de la boundary Discord
Callback Discord	Au cas par cas	Contexte ou règle non visible dans le décorateur
UI / Modal	Oui si comportement non trivial	Ownership, submit, contraintes Discord
Helper privé	Seulement si subtil	Convention ou algorithme
Test	Non par défaut	Le nom du test doit raconter le scénario

11. Cas particuliers à documenter après le refactoring erreurs / logs

La V1.1 va probablement modifier les contrats les plus sensibles. Les docstrings de ces zones doivent être revues **dans le même changement que le code**, pas dans une passe séparée plusieurs semaines plus tard.

Executors de rôles

À documenter une fois stabilisés :

- ce qui est garanti par le preflight ;
- ce qui ne peut pas être atomique côté Discord ;
- comment un état partiellement appliqué est signalé ;

- résultat retourné en succès ;
- exceptions que le coordinator doit traiter.

Reporting

À documenter :

- différence entre log technique et report administratif ;
- garantie qu'un reporter défaillant ne bloque pas les autres si cette règle est conservée ;
- politique de sanitization ;
- présence ou non du traceback selon la destination.

Boundaries Discord

À documenter lorsque le comportement est stabilisé :

- quelles exceptions sont capturées localement ;
- lesquelles sont laissées remonter vers une boundary commune ;
- comportement utilisateur en cas d'échec ;
- corrélation avec les logs/reports.

Base de données et transactions

Vérifier que les docstrings continuent d'indiquer clairement :

- qui ouvre la transaction ;
- qui commit ;
- qui rollback ;
- quelles méthodes sont conçues pour participer à une transaction externe.

12. Plan d'application

Phase 0 - avant le pilote live

Objectif : ne pas retarder le déploiement pour de la documentation cosmétique.

- commiter cette convention ;
- ne pas lancer de campagne de docstrings à 100 % ;
- corriger uniquement une docstring si elle est manifestement fausse ou trompeuse ;
- conserver les docstrings existantes qui décrivent correctement les contrats actuels.

Phase 1 - pendant le refactoring V1.1

Règle de contribution :

toute modification d'un contrat public ou d'un invariant important met à jour la docstring correspondante dans le même commit ou la même pull request.

Les chantiers erreurs / logs / reporting sont prioritaires.

Phase 2 - après stabilisation V1.1

Réaliser une passe documentaire ciblée :

1. inspecter les classes et méthodes publiques ;
2. identifier les docstrings absentes, trop vagues ou devenues fausses ;
3. compléter les contrats importants ;
4. supprimer les docstrings purement redondantes si elles ajoutent du bruit ;
5. vérifier la cohérence avec le guide pédagogique et la documentation d'architecture.

Phase 3 - lorsque le projet devient une API réutilisée

Si Clavier expose plus tard ses services à plusieurs consommateurs (Discord, Web, AI Core), envisager une documentation API **générée automatiquement** à partir du code et des docstrings.

Cette étape peut utiliser un outil adapté comme MkDocs + mkdocstrings, Sphinx ou pdoc. Elle ne doit pas créer une seconde copie manuelle de la documentation des fonctions.

13. Linting documentaire

Ruff est déjà utilisé pour le linting du projet, mais les règles **D** de pydocstyle ne sont pas actuellement activées.

Il n'est pas recommandé de les activer immédiatement avant le pilote live uniquement pour imposer une couverture artificielle. Cela créerait beaucoup de bruit sans améliorer la fiabilité fonctionnelle.

Après V1.1, l'équipe pourra décider d'activer progressivement un sous-ensemble de règles documentaires si elles correspondent réellement à la convention du projet.

La priorité reste : **qualité des contrats documentés > nombre de docstrings**.

14. Checklist de revue d'une docstring

Avant de valider une docstring importante :

- ☐ Explique-t-elle quelque chose que le nom et le typage n'expliquent pas déjà ?
- ☐ Décrit-elle le contrat plutôt que l'algorithme actuel ?
- ☐ Mentionne-t-elle un invariant important ?
- ☐ Les effets de bord externes sont-ils compréhensibles ?
- ☐ Les exceptions documentées existent-elles réellement ?
- ☐ Évite-t-elle de promettre une fonctionnalité future ?
- ☐ Reste-t-elle exacte si l'implémentation interne est refactorisée ?
- ☐ Ne contient-elle aucune donnée personnelle, secret ou identifiant réel ?
- ☐ Est-elle cohérente avec les autres docstrings du même composant ?
- ☐ Sa longueur est-elle proportionnée à la complexité du contrat ?

15. Règle de décision rapide

Lorsqu'un développeur hésite à écrire une docstring, poser cette question :

Si cette fonction est modifiée dans six mois, quelle information non évidente faut-il absolument connaître pour ne pas casser son contrat ?

S'il existe une réponse utile, elle appartient probablement à la docstring.

S'il n'existe aucune réponse autre que « la fonction fait ce que son nom indique », la docstring est probablement inutile.

16. Résumé de la convention

Claviger adopte une documentation du code **sélective et contractuelle** :

- le code et les types expliquent le fonctionnement immédiat ;
- les docstrings expliquent les responsabilités, invariants, effets de bord et erreurs importantes ;
- les documents d'architecture expliquent les décisions globales ;
- les tests démontrent les comportements ;
- une documentation API exhaustive ne sera générée que si le projet en a réellement besoin.

La V1 candidate possède déjà une base de docstrings importante. Le travail pertinent n'est donc pas de poursuivre un chiffre de couverture, mais de s'assurer que les composants critiques restent **compréhensibles, modifiables et sûrs à appeler** après les évolutions de la V1.1.